5. Project Development Phase

Introduction

The development phase involved implementing the complete machine learning workflow, the normalized ER data model, full user authentication, and deployment of the crop recommendation model using Flask.

Development Steps

Step 1: Environment Setup

Installed required libraries:

- NumPy
- Pandas
- SciPy
- Matplotlib
- Seaborn
- Scikit-learn
- Flask
- Werkzeug
- Joblib
- ReportLab
- Requests

Step 2: Dataset Collection

Downloaded the Crop Recommendation dataset (2200 records, 22 crop classes) and stored it as dataset/Crop_recommendation.csv.

Step 3: Data Preprocessing

- Loaded dataset using Pandas, coerced feature dtypes, dropped duplicates (src/preprocessing.py)
- Handled missing values (dropped rows with missing values after reporting counts)
- Applied feature scaling (StandardScaler)
- Encoded crop labels (LabelEncoder)

Step 4: Data Visualization (EDA)

Created plots via src/eda.py, stored under static/images/eda/, including missing values per column, correlation heatmap, feature distribution histograms, class balance, pairwise feature relationships, confusion matrix and feature importance.

Step 5: Model Development

Algorithms Compared (src/train_model.py):

- Random Forest Classifier
- Logistic Regression
- K-Nearest Neighbors (KNN)

- Support Vector Classifier (SVC)
- Gaussian Naive Bayes

Performed an 80/20 stratified train-test split, trained all five models, and compared test accuracy and macro F1-score. The best-performing model was kept automatically and persisted.

Step 6: Model Saving

Saved the trained artifacts using Joblib:

- model/crop_model.joblib
- model/scaler.joblib
- model/label_encoder.joblib
- model/metrics.json (accuracy, macro-F1, per-model scores, confusion matrix, feature importance).

Step 7: Flask Web Application

Created:

app.py

Templates

Implemented prediction Funtionality.

Project Structure

- database/
- dataset/
- model/
- notebooks/
- src/
- static/
- templates/
- app.py
- auth.py
- database.py
- model_training.py
- README.md
- requirements.txt
- utils.py

Output

The Flask application accepts user input (once logged in) and recommends the most suitable crop with a confidence score, while recording a full SoilData → Prediction → Report trail.

Conclusion

The development phase successfully integrated machine learning, a normalized ER data model, and full authentication into a complete, secure crop recommendation system.